EECS 492 - Homework 1 (Winter 2026)

Due: Wednesday, February 4, 2026 at 23:59

Logistics

Submission Guidelines

You must work alone for the conceptual problems, though you may discuss ideas and high-level concepts with classmates. You may optionally work with a partner on the coding sections, though you can work alone if you want.

The due date for this assignment is **Wednesday, February 4, 2026 at 23:59**. The submission is tracked using Gradescope. The written section and programming section are graded and penalized separately. Also note that any change you make to the homework already submitted on Gradescope counts as a resubmission. If you make any changes to the assignment after the due date has passed you will be assigned a late penalty based on the number of days that have passed. For example, if you edit an assignment on February 6 and it was due on February 3 you will be assigned a 30% penalty (10% per day). This is non-negotiable.

You must submit the following files to Gradescope (note that there are two separate Gradescope submissions):

1. **A PDF containing the solutions to the written section**. You can write your solutions by hand, use a tablet, or typeset your solutions in LaTeX. We only ask that you make it easy to read and not too verbose so that graders don't struggle to understand your writing. You must submit the written section to the Gradescope individually.
2. **The completed `dt.py` and `hw1.ipynb` file to the programming section**. If you work with a partner, you must put both people's names on the Gradescope submission.

Late Day Tokens

To accommodate for coinciding deadlines you may have from other courses, or personal unforeseen events such as sickness, each person has 3 slip day tokens. Each token provides an automatic extension of 1 calendar day—no questions asked.

1. Each late day token covers the whole homework (both written part and programming part) for extension of 1 calendar day. For example, if you submitted written part 1 day late and programming part 2 days late. Then, if you use 1 late day token, you don't have any penalty on the written part, but will have 10% penalty on the programming part.
2. Each homework has a Gradescope submission for the late day token, in addition to the written part and programming part.

3. Late days are rounded up to the nearest integer. For example, a submission that is 4 hours late will count as one day late.
4. Extreme circumstances, like medical emergencies, etc.: In such cases, additional, no-penalty extensions will be granted. Contact your section faculty instructor with some written documentation (like doctor's note).
5. Emails requesting exceptions for other reasons will be ignored with no reply.
6. Homework turned in after three days will not be accepted.

Grading Policy

You have one week to submit a regrade request once the grades for your homework are out. We will provide solutions, along with the exact grading rubric used.

Collaboration

Collaborating with people is encouraged, but plagiarism is strictly prohibited. As mentioned in **Submission Guidelines**, you must work alone for the conceptual problems, though you may discuss ideas and high-level concepts with classmates. You may optionally work with a partner on the coding sections, though you can work alone if you want.

Generative AI

Learning how to use AI functions such as ChatGPT is important for all of us. Used properly, ChatGPT can enhance our work; used improperly, it can border on plagiarism. You are allowed to use ChatGPT or similar only for conceptual questions, and **it is strictly forbidden for programming**. If you have used ChatGPT on anything you submit for EECS 492, please include an explanation as to:

- State which model you used
- Include the prompt(s) that you gave the model
- Explain your reasoning as to why you agree (or disagree) with the response
- Mention any incorrect responses or erroneous behavior that you may have seen

1 Written [50 Points]

1.1 Designing Agents [10 Points]

Tic-tac-toe is a game for two players who take turns marking the spaces in a three-by-three grid with X or O. The game's objective is to be the first in placing three of their markers in a horizontal, vertical, or diagonal row (see Figure 1).

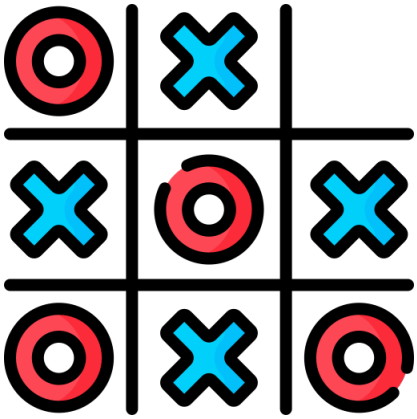

Figure 1: A game of Tic-Tac-Toe where the O player has won, as it has three markers in a diagonal row.

You are now tasked with developing an AI algorithm for a tic-tac-toe agent. The agent is a robot that can play against another agent (human or robot) on a piece of paper using a pen. The agent must be capable of:

- detecting whether a marker is a X or O
- drawing a marker in one of the nine grids
- detecting the empty grids and markers already present on the paper
- using this information to determine if it has won, lost, or tied the game

Note that the agent is not just playing one time, but rather, is a robot that is supposed to be able to play multiple games as many times as needed. Now, answer the following questions - note that some of these cases might be ambiguous, so justify your answers and explicitly state any assumptions

(a) [5 Points] Create the PEAS description for the task environment, where the task environment includes all the games the agent plays.

(b) [5 Points] Identify the characteristics of the environment and provide a brief justification as to why you characterized the environment in that way
As a reminder, the characteristics of the environments are as follows:

- Fully observable vs. partially observable
- Single-agent vs. multi-agent
- Deterministic vs. nondeterministic
- Episodic vs. sequential
- Static vs. dynamic
- Discrete vs. continuous
- Known vs. unknown

1.2 K-Nearest Neighbors [10 Points]

The training dataset below contains 6 data points (see Table. 1). Each data point has 3 features and a class name (A/B). We want to use a k-nearest neighbors (k-NN) classifier to predict the class of any new testing data point (query point). Note that, the distance between 2 data points is the Minkowski distance (L^p norm). In this homework, we use $p = 2$ for Minkowski distance, which is the Euclidean distance. Recall that, the Euclidean distance between a and b for n-dimensional space is $d = \sqrt{(a_1 - b_1)^2 + (a_2 - b_2)^2 + \cdots + (a_n - b_n)^2}$.

x	`Feature 1`	`Feature 2`	`Feature 3`	Class
x_1	5	2	8	$y_1 = \text{A}$
x_2	7	6	1	$y_2 = \text{B}$
x_3	0	6	0	$y_3 = \text{A}$
x_4	2	7	4	$y_4 = \text{B}$
x_5	9	2	4	$y_5 = \text{A}$
x_6	3	5	8	$y_5 = \text{B}$

Table 1: Training dataset for k-nearest neighbors

Now, we have a query point with `Feature 1` = 8, `Feature 2` = 0, `Feature 3` = 3. Suppose we only consider `Feature 1` to `Feature m` for Trial m. For example, we only consider `Feature 1` and `Feature 2` for Trial 2. Calculate the distances from the query point to the training data points (round to 3 decimal places) for Trial 1 to 3 and filling the blanks in the table below (see Table. 2). Then, run the k-NN algorithm with $k = 3$ and write the 3 nearest neighbors (e.g. x_1, x_2, x_3) as well as the classification of the query point for each trial.

x	Trial 1	Trial 2	Trial 3
x_1			
x_2			
x_3			
x_4			
x_5			
x_6			

Table 2: Distances table of k-NN

1.3 Decision Tree [15 Points]

Given the training dataset below (see Table. 3), we will use the decision tree to classify whether a team can play Basketball(`PlayBasketball` = Yes) or not PlayBasketball (`PlayBasketball` = No). There are three attributes for each data point, namely `Temperature`, `Humidity`, and `Outlook`. `Temperature` can take from $\{Hot, Mild, Cool\}$. `Humidity` can take from $\{High, Normal\}$. `Outlook` can take from $\{Sunny, Overcast, Rain\}$.

x	`Outlook`	`Humidity`	`Temperature`	`PlayBasketball`
x_1	Sunny	High	Hot	y_1 = No
x_2	Sunny	High	Hot	y_2 = No
x_3	Overcast	High	Hot	y_3 = Yes
x_4	Rain	High	Mild	y_4 = Yes
x_5	Rain	Normal	Cool	y_5 = Yes
x_6	Rain	Normal	Cool	y_6 = No
x_7	Overcast	Normal	Cool	y_7 = Yes
x_8	Sunny	High	Mild	y_8 = No
x_9	Sunny	Normal	Cool	y_9 = Yes
x_{10}	Rain	Normal	Mild	y_{10} = Yes

Table 3: Training dataset for decision tree learning

(a) [6 Points] Calculate the information gain for each attribute and answer what should be the first splitting attribute (root node)? (round to 3 decimal places)

(b) [6 Points] Assume that you decide to first split on Humidity. Consider the Two branches that result.

(i) What is the resulting entropy of the class label given the data points in each of the Two branches?(round to 3 decimal places)

(ii) What is the Information Gain (IG) for Humidity? (Note, you calculated this in problem 1.3a, just write the value here).(round to 3 decimal places)

(iii) Assume that you next split on Temperature. What is the resulting IG for Temperature on each branch?(round to 3 decimal places)

(c) [3 Points] List and briefly explain two distinct problems of using decision trees for classification.

1.4 Regression [15 Points]

(a) [8 points] We want to train an univariate linear regression model given the training dataset in Table 4. Our model will take the form $h_{\mathbf{w}}(x) = w_1 x + w_0$. What values of w_0 and w_1 minimize the empirical loss? For reference, our loss function is below.

$$\begin{aligned}\mathcal{L}oss(h_{\mathbf{w}}(x)) &= \sum_i L_2(y_i, h_{\mathbf{w}}(x_i)) \\ &= \sum_i (y_i - h_{\mathbf{w}}(x_i))^2\end{aligned}$$

Input feature x	Output y
$x_1 = 7$	$y_1 = 6$
$x_2 = 2$	$y_2 = 2$
$x_3 = 4$	$y_3 = 3$

Table 4: Training dataset for linear regression

(b) [2 points] Now, suppose that we receive two new data points, given in Table 5.

Input feature x	Output y
$x_4 = 3$	$y_4 = 2$
$x_5 = -1$	y_5 = -1

Table 5: Testing dataset for linear regression

Using the weights that you calculated in part (a), what values will your model predict for x_1 through x_5? Your answer should list all five predictions.

(c) [3 points] Given all five data points, what is the overall loss of your model? Use the loss function given above.

(d) [2 points] Compute the R^2 statistic for your model. What does this tell you about the effectiveness of your model?

2 Programming [50 Points]

2.1 Setup

All programming assignments for this course use Python. If you're new to Python or not very experienced with it, please come to office hours; we're happy to help! We recommend also checking out the Python Cheatsheet. The topics you'd need from here are: Basics, Built-in Functions, Control Flow, Functions, Lists, Tuples, Dictionaries, Sets, OOP, and Virtual Environments. We recommend also checking out the Python Resources document on Canvas. We have also shared a document Debugging Tips to help you debug your code. These resources are on Canvas, under 'Files' > 'Coding Resources'.

2.1.1 Google Colab Setup

To avoid issues with Python installations and virtual environments, the code for this assignment will be completed in **Google Colab**. We recommend these steps for getting started:

(1) Download the starter code files off of Canvas.

(2) Create a folder for this HW1 programming assignment on your Google Drive and upload all of the starter code files into that folder.

(3) One of the starter code files is a notebook `.ipynb` file. To open this file, create a blank Google Colab notebook, go to 'File' > 'Open Notebook', and select the 'hw1.ipynb' starter code file. You can now run and edit this notebook to complete the assignment!

(4) The Colab notebook has additional directions for making sure the notebook can read from the other relevant starter code files.

Please come to office hours if you have any issues running in Colab!

2.2 Problem Statement

In this assignment, you will implement core components of a **decision tree classifier from scratch**. The goal is to understand how decision trees split data using **entropy** and **information gain**.

We use the **Banknote Authentication dataset**, provided as `bank_auth.txt`. The dataset has 4 continuous features and a binary label in {0,1} as the last column.

You are given two files with **four** TODO blocks:

- **`dt.py`**: decision tree implementation. (Question 2.3)
- **`hw1.ipynb`**: loads data, trains the model, evaluates accuracy and further discussion. (Question 2.5 and Question 2.6)

Do not modify any code outside the TODO blocks.
Submit the two files to Gradescope.

2.3 Decision Tree Implementation (`dt.py`)

The decision tree is already fully implemented except for two functions.

2.3.1 TODO 1: Entropy [15 points]

Implement:

```
def entropy(y: np.ndarray) -> float:
```

This function should compute:

$$H(y) = -\sum_{c \in \{0,1\}} p_c \log_2 p_c$$

Use the convention that $0 \log 0 = 0$.

2.3.2 TODO 2: Information Gain [15 points]

Implement:

```
def information_gain(y, y_left, y_right) -> float:
```

$$IG = H(y) - \frac{|L|}{|y|} H(y_L) - \frac{|R|}{|y|} H(y_R)$$

These functions are used internally when selecting the best split.

2.4 How Continuous Features Are Split

Note: This subsection is purely explanatory. There is nothing to compute or implement here.
Although features are continuous, the decision tree always makes **binary decisions**. At each node, the algorithm:

- considers candidate thresholds between sorted unique feature values,
- splits data using the rule:

$$x_j \leq \tau \quad \text{vs.} \quad x_j > \tau$$

The threshold that yields the highest information gain is selected.

2.5 Training and Evaluation (`hw1.ipynb`)

The notebook:

1. loads `bank_auth.txt`,
2. creates a deterministic 80/20 train–test split,
3. trains the decision tree,
4. reports training and test accuracy,
5. prints the learned tree.

TODO (in the notebook): Train a new decision tree on the same dataset using a different train/test split.

Answer in hw1.ipynb:
Report the training and test accuracy for both decision trees (a correct implementation should achieve over 90% accuracy). Compare the printed tree structures—are they different? Explain why or why not. [10 points]

2.6 Question: Effect of Tree Depth [10 points]

Decision trees can underfit or overfit depending on their depth.

TODO (in the notebook): Train decision trees with:

$$\texttt{max_depth} \in \{1, 2, 4, 8\}$$

and record the training and test accuracy for each.

Answer in hw1.ipynb (2–3 sentences):
Report the training and test accuracy for each. How does increasing tree depth affect training accuracy and test accuracy? Why?